GPU-accelerated Plane Graph Region Extraction Algorithms

Anonymous Author(s)
Department of Computer Science
The American University in Cairo
Email: author@example.com

***Abstract*—This paper presents two high-performance GPU-accelerated algorithms for the extraction of connected regions in plane graphs using the NVIDIA CUDA parallel programming model. While traditional sequential algorithms for extracting all regions of a plane graph $G(V, E)$ achieve an optimal asymptotic time complexity of $O(m \log m)$, their throughput remains bounded by CPU core frequencies when scale transitions into massive real-world engineering workloads. Existing geometric data structure libraries, such as the Boost Graph Library (BGL) and the Computational Geometry Algorithms Library (CGAL), provide stable CPU baselines but lack optimized parallel implementations for modern many-core architectures. To address this computational bottleneck, we introduce a highly parallelized framework utilizing edge orientation sorting, pointer jumping for region leader identification, and optimized face traversal on the GPU. Experimental evaluations across various large-scale synthetic mesh layouts demonstrate that our CUDA implementations achieve absolute correctness and deliver up to a $33\times$ speedup compared to state-of-the-art CPU baselines.**

***Index Terms*—Plane Graph, Region Extraction, Computational Geometry, Meshing, GPU, CUDA, Parallel Algorithms**

I. Introduction

A plane graph is a specific topological embedding of a planar graph in a two-dimensional Euclidean space, where vertices are mapped to distinct coordinates and edges are mapped to continuous arcs that intersect exclusively at their common endpoints. Identifying and extracting the discrete regions (or faces) bounded by these edges is a fundamental operation across an extensive array of domains, including geographic information systems (GIS), computer graphics, VLSI layout verification, mesh generation, and automated robotics navigation [**?**], [**?**], [**?**]. In modern computational engineering, simulations involving finite element analysis or fluid dynamics frequently manipulate structural meshes containing tens of millions of distinct elements [**?**], [**?**], [**?**]. At this scale, serial or poorly parallelized geometric extraction routines quickly become severe computational bottlenecks.

In classical sequential computational geometry, two primary frameworks provide optimal paradigms for region extraction. The first approach utilizes directional angular sweeps or "wedges" around vertices to trace boundaries [**?**]. The second approach relies on adjacency topologies, specifically the Half-Edge (HE) or Doubly Connected Edge List (DCEL) structures, which link directed edge representations to form facial walks [**?**]. Both formulations achieve an optimal sequential time complexity of $O(m \log m)$ and a space complexity of $O(m)$, where m is the number of edges in the graph $G(V, E)$. According to Euler's polyhedral formula for planar graphs, any simple plane graph with $n \geq 3$ vertices is inherently sparse, satisfying the property $m \leq 3n - 6$ [**?**].

Despite this linear structural scale, the global sorting steps inherent to vertex-centered angle computations restrict sequential performance. Massively parallel architectures, such as NVIDIA's CUDA platform [**?**], offer an architectural path to accelerate these operations by executing edge sorting and pointer tracing across thousands of hardware threads simultaneously.

This paper introduces comprehensive GPU-accelerated parallel adaptations of both the Wedge-based and the Half-Edge-based region extraction algorithms. We evaluate our parallel CUDA C++ implementations against established CPU geometric frameworks, including CGAL [**?**] and BGL [**?**], as well as heavily optimized sequential baselines. Our parallel designs leverages the parallel primitives of the Thrust library [**?**] and adapts a parallel pointer-jumping (path doubling) mechanism. The results indicate that the proposed GPU architectures scale effectively with graph density, outperforming conventional single-threaded processors by up to $33\times$ while ensuring robust numerical and topological correctness.

The remainder of this work is structured as follows. Section II formalizes the necessary mathematical definitions and structural notations. Section III outlines the operational logic of the baseline sequential algorithms. Section IV provides a deep overview of our proposed parallelization methodologies and CUDA implementation strategies. Section V presents the experimental results, performance characteristics, and workload profiling, and Section VI concludes the paper.

II. Definitions and Notations

Planar Graph vs. Plane Graph: It is critical to differentiate between the structural topology of a graph and its spatial realization. A graph $G = (V, E)$ is classified as *planar* if it possesses the inherent topological capacity to be mapped onto $\mathbb{R}^2$ without edge crossings. Conversely, a *plane graph* refers specifically to a fixed, crossing-free planar embedding of a planar graph. As stated by Kuratowski's Theorem, non-planar graphs are characterized by containing subgraphs homeomorphic to the complete graph K_5 or the complete bipartite graph $K_{3,3}$, both of which possess an unavoidable crossing number

greater than zero under any geometric arrangement in the plane (see Fig. 1).

(a) The complete graph K_5 (b) The bipartite graph $K_{3,3}$

Fig. 1: The two minimal non-planar graphs depicted with unavoidable edge crossings.

A planar graph can be rendered in a non-plane fashion if its vertices are positioned poorly. For example, the complete graph on four vertices, K_4, is planar; however, a standard rectilinear drawing may generate intersecting diagonals, whereas a valid plane embedding rearranges the vertices or routes edges externally to enforce an intersection-free state, as shown in Fig. 2.

(a) A plane embedding of K_4 (b) A non-plane drawing of K_4

Fig. 2: Geometric embeddings of K_4: representation (a) constitutes a true plane graph, whereas representation (b) features a central edge crossing.

Regions and Wedges: A plane graph partitions the continuous two-dimensional plane into a set of bounded open regions, denoted as faces or meshes, alongside a single unbounded exterior region. Every face is topologically bounded by a closed cycle of directed edge steps. Consider the plane graph illustrated in Fig. 3. It consists of six vertices and seven edges, which partition the plane into two bounded interior regions (R_1 and R_2) and one unbounded exterior region (R_3):

$$R_1 = \langle (v_1, v_2), (v_2, v_4), (v_4, v_5), (v_5, v_6), (v_6, v_1) \rangle$$
$$R_2 = \langle (v_1, v_6), (v_6, v_3), (v_3, v_1) \rangle$$
$$R_3 = \langle (v_1, v_3), (v_3, v_6), (v_6, v_5), (v_5, v_4), (v_4, v_2), (v_2, v_1) \rangle$$

A *wedge* represents the angular sector centered at a vertex bounded by two consecutive incident edges in a clockwise order. Formally, for an undirected edge pair sharing vertex v_j, if rotating edge (v_i, v_j) clockwise around v_j reaches edge (v_j, v_k) without crossing any intermediate incident edges, then the ordered triplet (v_i, v_j, v_k) defines a valid wedge. For any plane graph containing m edges, exactly $2m$ distinct wedges exist. Two wedges are defined as *contiguous* if the second wedge begins with the directed edge that completes the first wedge.

Half-Edges and Twin Pointers: In directional data structures, every undirected edge is decomposed into two distinct, oppositely oriented components termed *half-edges*. For a half-edge $e = \langle u, v \rangle$ starting at source vertex u and terminating at target vertex v, its associated *twin half-edge* is defined as $\bar{e} = \langle v, u \rangle$ (see Fig. 4). Each half-edge stores structural references, most notably a pointer to its immediate successor (`next`) along the perimeter of the incident region.

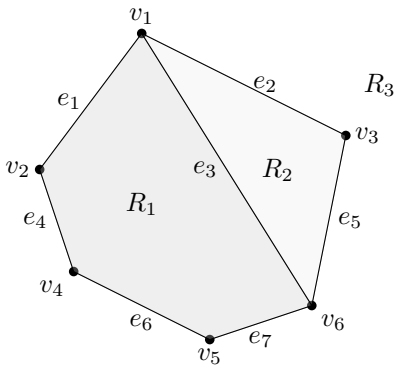

Fig. 3: A sample plane graph comprising six vertices and seven edges forming three discrete regions.

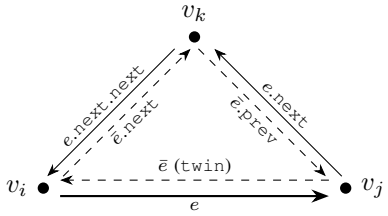

Fig. 4: Topological orientation of half-edges and twin pairings forming a face boundary.

III. Optimal Sequential Algorithms

To establish context for the parallel adaptations, we first review the computational pipeline of the two optimal $O(m \log m)$ serial extraction algorithms.

A. The Wedge-based Algorithm

This formulation segments the extraction task into an independent angular construction phase followed by an iterative tracing routine [**?**].

1) Phase 1: Wedge Construction:

1) **Edge Duplication:** Every undirected edge (v_i, v_j) is split into two directed instances, $\langle v_i, v_j \rangle$ and $\langle v_j, v_i \rangle$, requiring $O(m)$ time.
2) **Angle Calculation:** For each directed instance, the absolute polar angle $\theta \in [0, 2\pi)$ relative to a positive horizontal vector centered at its source vertex is computed in $O(m)$ time.
3) **Angular Sorting:** The array of directed edges is sorted using a primary key of the source vertex ID and a secondary key of the polar angle θ. This organizes all outgoing edges around each vertex in a strict counterclockwise sequence in $O(m \log m)$ time (see Fig. 5).

4) **Wedge Generation:** For a sorted sequence of outgoing edges $\langle e_0, e_1, \ldots, e_{d-1} \rangle$ incident to vertex v_i, consecutive pairs are combined to form wedges. Specifically, edge $e_k = \langle v_i, v_j \rangle$ and $e_{k+1} = \langle v_i, v_k \rangle$ yield the wedge (v_k, v_i, v_j), with the final edge looping back to the first. This requires $O(m)$ execution time.

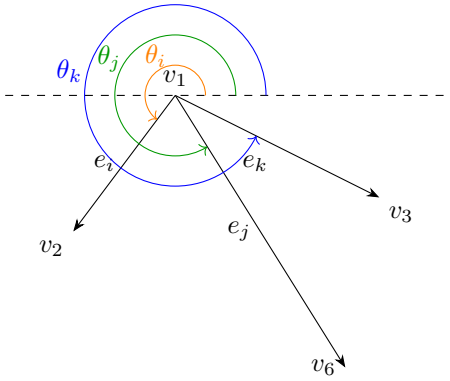

Fig. 5: Angular sorting of directed edges around a local hub vertex v_1.

2) Phase 2: Face Assembly: A region is structurally equivalent to a closed sequence of contiguous wedges.

1) **Wedge Lexical Sorting:** The generated wedges (v_k, v_i, v_j) are sorted by their first two vertex identifiers (v_k followed by v_i) in $O(m \log m)$ time.
2) **Tracing Cycles:** The algorithm iterates through the sorted list. Unvisited wedges serve as seeds for new regions. For a current wedge (v_k, v_i, v_j), its unique contiguous successor must begin with the sequence v_i, v_j. The algorithm locates this successor in $O(\log m)$ time via binary search over the sorted wedge list. Tracing continues until the seed wedge is encountered again, sealing the region. The cumulative cost across all $2m$ elements is bounded at $O(m \log m)$ time.

B. The Half-Edge-based Algorithm

The half-edge framework replaces explicit wedge searching with structural pointer records.

1) Phase 1: Half-Edge Topology Construction: Similar to the wedge method, undirected edges are duplicated into twin half-edges and sorted by source vertex and polar angle in $O(m \log m)$ time. Next, explicit connectivity links are established: for a half-edge $e_{i,j}$ terminating at vertex v_j, its immediate clockwise neighbor in the sorted edge rotation around v_j is retrieved. The `next` pointer of the twin half-edge $\bar{e}_{i,j}$ is assigned directly to this neighbor. This step links half-edges into a continuous chain bordering a face, completing in $O(m)$ time.

2) Phase 2: Half-Edge Traversal: Because every half-edge explicitly points to its unique face successor via the `next` pointer, region extraction collapses into an $O(m)$ linear traversal. The algorithm iterates through the half-edge array, tracking unvisited items and following the `next` references until a cycle closes.

IV. GPU-Accelerated Parallel Framework

Transitioning these approaches to a highly parallel GPU computing architecture requires eliminating sequential tracking dependencies and mitigating thread workload imbalances.

A. Parallel Angle Sorting and Data Layout

Both parallel algorithms begin by loading the graph data into flat, contiguous device vectors allocated on the GPU. We avoid pointer-heavy structures to ensure coalesced global memory access patterns. Edges are duplicated in parallel using a simple CUDA kernel where each thread handles one undirected edge from the input stream.

To execute the critical vertex rotation sort on the GPU, we utilize the primitive sorting functions provided by the Thrust library [**?**]. Rather than executing thousands of isolated local sorts for each vertex, which would cause significant warp divergence, we perform a single global radix sort across the entire duplicated edge array. A compound 64-bit key is constructed: the most significant 32 bits store the source vertex ID, while the least significant 32 bits contain the floating-point polar angle θ. Thrust's `sort_by_key` reorders the global edge array in a single operation, grouping edges by vertex and ordering them counter-clockwise.

B. Cycle Identification via Parallel Pointer Jumping

The primary challenge in parallelizing Phase 2 of both algorithms is eliminating the sequential dependency when tracing a cycle. If each thread simply walks a face sequentially, threads assigned to small triangular regions will finish instantly and idle, while threads assigned to complex regions with many vertices will create long-running tails. Furthermore, threads must safely determine unique face leadership without race conditions or expensive atomic locks.

To break this sequential bottleneck, we utilize a parallel pointer jumping (or pointer aliasing) technique. Initially, every wedge (or half-edge) i is assigned a pointer to its next contiguous neighbor, $S_0[i] = \texttt{next\_id}$, and its face leader label is initialized to its own index, $L_0[i] = i$.

In successive parallel iterations, every element updates its pointer and its leader label concurrently by tracking its successor's state:

$$S_{t+1}[i] = S_t[S_t[i]]$$
$$L_{t+1}[i] = \min\left(L_t[i], L_t[S_t[i]]\right)$$

This process updates pointers across the graph in parallel. For a face containing a cycle of length c, the pointer jumping updates converge in $\lceil \log_2 c \rceil$ steps. Upon convergence, all elements belonging to the same closed region point to a single terminal node, and their labels collapse to the minimum index within that cycle, which designates the unique region leader.

Fig. 6 illustrates this propagation through a discrete 8-element cycle. At $t = 0$, each element points to its immediate predecessor. By $t = 2$, elements have skipped forward, quickly distributing the minimum index across the array. By iteration 3 ($\log_2 8$), the entire cycle converges to the common leader label 0.

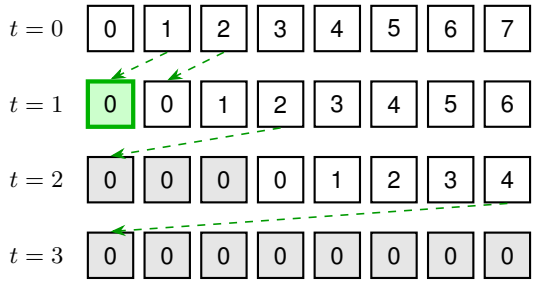

Fig. 6: Convergence of parallel pointer jumping on an 8-element cycle, showing label propagation to the minimum index (0).

C. Linearization and Workload Balancing

Once the region leaders are identified, a final kernel runs to serialize the output paths for storage. In this phase, we change the parallelization strategy: instead of launching one thread per edge or wedge ($2m$), we launch exactly one thread per unique region leader ($|R|$).

Each region thread performs a sequential walk through its assigned cycle, writing the vertex IDs into a contiguous output buffer. While this step is sequential within each region, allocating threads at the region level reduces thread management overhead and prevents resource over-subscription on the GPU. However, this approach is sensitive to variations in region size; the overall execution time of this kernel is determined by the largest cycle in the graph.

An alternative approach would be to completely linearize the output using parallel prefix sums on the pointer-jumped vectors. However, allocating threads based on $|R|$ avoids the need for massive auxiliary tracking vectors. As a result, this strategy runs faster in practice because it reduces kernel launch overhead and improves memory access patterns.

V. Experiments and Results

A. Hardware and Baseline Configurations

The algorithms were evaluated on a high-performance computing node equipped with a 24-core Intel Xeon Gold 6548Y+ processor (5th Generation, 2.50 GHz base frequency, 4.10 GHz Turbo) and an NVIDIA A100-SXM4 GPU with 80 GB of HBM2e VRAM, 108 streaming multiprocessors, and 6,912 CUDA cores.

Our proposed CUDA C++ algorithms were compared against four CPU configurations:

1) **BGL (Boost Graph Library):** Uses the `boyer_myrvold_planarity_test` routine [**?**] to build a planar embedding before extracting faces. This planarity verification step introduces a large runtime overhead, making BGL impractical for massive graphs where planarity is already guaranteed. To ensure a fair comparison, we report only the face extraction runtime after validation, and only for the smallest graph instances.
2) **CGAL (Computational Geometry Algorithms Library):** A robust geometry library [**?**]. CGAL processes input segments by building a planar arrangement, automatically computing intersections to split the plane into non-overlapping regions. This generality makes it robust but introduces additional processing overhead compared to specialized mesh extractions.
3) **Optimized CPU Wedge Baseline:** Our single-threaded C++ implementation of the sequential wedge algorithm [**?**].
4) **Optimized CPU HE Baseline:** Our single-threaded C++ implementation of the sequential half-edge algorithm.

B. Dataset Patterns

The evaluation used three synthetic 2D mesh patterns generated over an $l \times l$ grid of vertices, with $l \in \{1001, 4001, 8001, 12001\}$.

- **Pattern 1 (P_1):** A dense mesh featuring horizontal, vertical, and alternating diagonal edges. This layout forms a uniform grid of small triangular faces, bounded by a single large exterior region of size $4(l-1)$.
- **Pattern 2 (P_2):** A sparser structure where selective diagonal deletions form a mix of small triangles and large zigzag regions.
- **Pattern 3 (P_3):** A hybrid configuration combining regions of varying shapes and sizes to create a mid-density mesh.

C. Performance Evaluation

Table I summarizes the execution runtimes across twelve representative test graph instances (g_1 to g_{12}), covering all combinations of sizes and structural patterns.

The GPU implementations scale efficiently as the graph size increases. For small graphs (g_1 to g_3), the GPU speedup is limited by kernel launch overhead and device synchronization costs. However, on large workloads (g_{10} to g_{12}), the GPU framework balances processing across its core architecture, achieving a maximum speedup of $33.17\times$ on g_{11}. Across all large-scale instances, the average speedup matches $23.7\times$.

Fig. 7 highlights a performance drop for Pattern 1 (P_1) topologies, seen in instances g_7 and g_{10}. This behavior stems from load imbalances during the sequential region linearization step. The algorithm's parallel efficiency depends on the maximum region size in the mesh. Because Pattern 1 is dominated by a single large exterior region ($size = 4(l-1)$) alongside many small triangular regions, the GPU threads processing smaller faces complete quickly and idle while waiting for the single thread processing the exterior region to finish. Higher speedups are achieved on patterns with homogeneous region sizes (P_2 and P_3), which ensure a balanced workload across all active threads.

Finally, comparing the two parallel approaches shows that the Half-Edge implementation consistently outperforms the Wedge-based framework. This advantage arises because the Half-Edge data structure tracks connectivity natively, avoiding

TABLE I: Performance comparison of region extraction algorithms (Runtimes in seconds).

Graph	Grid Size	Pattern	Edges (m)	GPU HE	GPU Wedge	CPU HE	CPU Wedge	BGL	CGAL
g_1	1001×1001	P_1	2,998,000	0.048	0.052	0.612	0.734	3.45	12.11
g_2	1001×1001	P_2	1,998,000	0.022	0.028	0.415	0.510	2.11	8.42
g_3	1001×1001	P_3	2,497,000	0.031	0.038	0.512	0.622	2.89	10.15
g_4	4001×4001	P_1	47,992,000	0.412	0.445	10.120	11.950	N/A	N/A
g_5	4001×4001	P_2	31,992,000	0.185	0.211	4.710	5.612	N/A	N/A
g_6	4001×4001	P_3	39,992,000	0.179	0.195	4.690	5.643	N/A	N/A
g_7	8001×8001	P_1	191,984,000	1.577	1.720	21.130	26.200	N/A	N/A
g_8	8001×8001	P_2	127,984,000	0.496	0.551	15.578	17.922	N/A	N/A
g_9	8001×8001	P_3	159,984,000	0.770	0.840	20.850	24.338	N/A	N/A
g_{10}	12001×12001	P_1	431,976,000	3.520	3.850	47.990	56.430	N/A	N/A
g_{11}	12001×12001	P_2	287,976,000	0.995	1.121	33.004	39.120	N/A	N/A
g_{12}	12001×12001	P_3	359,976,000	1.482	1.610	41.760	49.210	N/A	N/A

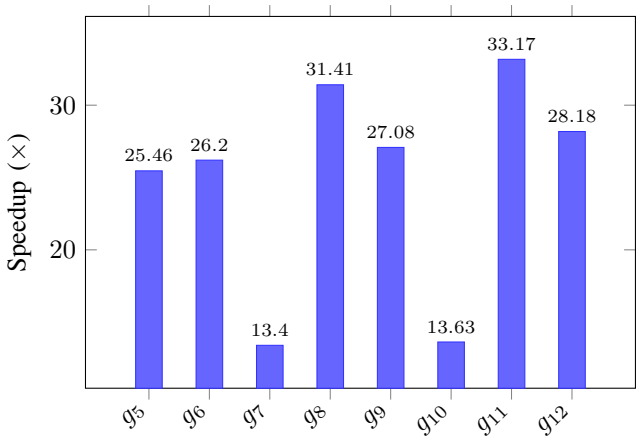

Fig. 7: Speedup of the GPU Half-Edge-based implementation compared to the single-threaded CPU baseline across large graph instances.

the secondary wedge-sorting step required by the wedge algorithm.

VI. Conclusion

This paper introduced two high-performance GPU-accelerated algorithms for region extraction in plane graphs using CUDA C++. Our parallel formulations optimize edge sorting and face traversal by combining the Thrust library with a parallel pointer jumping technique to determine region leadership without sequential tracking dependencies.

While both approaches share a sequential time complexity of $O(m \log m)$, the Half-Edge implementation runs faster in practice because it avoids the secondary sorting step required by the Wedge-based method. The proposed framework scales efficiently, achieving up to a $33\times$ speedup over optimized single-threaded CPU baselines on large graphs. This makes it highly suitable for high-throughput pipelines in GIS, VLSI design, finite element meshing, and computer graphics.

References

[1] P. Antolin, A. Buffa, and E. Cirillo, "Region extraction in mesh intersection," *Computer-Aided Design*, vol. 156, p. 103448, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0010448522001816

[2] S. V. G. de Magalhães, W. R. Franklin, and M. V. A. Andrade, "An efficient and exact parallel algorithm for intersecting large 3-d triangular meshes using arithmetic filters," *Computer-Aided Design*, vol. 120, p. 102801, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0010448519305330

[3] S. V. G. Magalhães, W. R. Franklin, and M. V. A. Andrade, "Fast exact parallel 3d mesh intersection algorithm using only orientation predicates," in *Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, ser. SIGSPATIAL '17. New York, NY, USA: Association for Computing Machinery, 2017. [Online]. Available: https://doi.org/10.1145/3139958.3140001

[4] A. Lintermann, *Computational Meshing for CFD Simulations*. Singapore: Springer Singapore, 2021, pp. 85–115. [Online]. Available: https://doi.org/10.1007/978-981-15-6716-2$_6$

[5] A. Lintermann, S. Schlimpert, J. Grimmen, C. Günther, M. Meinke, and W. Schröder, "Massively parallel grid generation on hpc systems," *Computer Methods in Applied Mechanics and Engineering*, vol. 277, pp. 131–153, 2014. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0045782514001340

[6] P. Cheng, L.-F. Mao, W.-H. Shen, and Y.-L. Yan, "Electromigration failures in integrated circuits: A review of physics-based models and analytical methods," *Electronics*, vol. 14, no. 15, 2025. [Online]. Available: https://www.mdpi.com/2079-9292/14/15/3151

[7] X. Jiang and H. Bunke, "An optimal algorithm for extracting the regions of a plane graph," *Pattern Recognition Letters*, vol. 14, no. 7, pp. 553–558, 1993. [Online]. Available: https://www.sciencedirect.com/science/article/pii/016786559390104L

[8] M. De Berg, M. Van Kreveld, M. Overmars, and O. Cheong, *Computational Geometry: Algorithms and Applications*, 3rd ed. Springer-Verlag Berlin Heidelberg, 2008.

[9] R. Diestel, *Planar Graphs*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2025, pp. 93–122. [Online]. Available: https://doi.org/10.1007/978-3-662-70107-2$_4$

[10] J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with cuda," in *ACM SIGGRAPH 2008 Classes*, ser. SIGGRAPH '08. New York, NY, USA: Association for Computing Machinery, 2008. [Online]. Available: https://doi.org/10.1145/1401132.1401152

[11] A. Fabri and S. Pion, "Cgal: the computational geometry algorithms library," in *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, ser. GIS '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 538–539. [Online]. Available: https://doi.org/10.1145/1653771.1653865

[12] *The boost graph library: user guide and reference manual*. USA: Addison-Wesley Longman Publishing Co., Inc., 2002.

[13] N. Bell and J. Hoberock, "Chapter 26 - thrust: A productivity-oriented library for cuda," in *GPU Computing Gems Jade Edition*, ser. Applications of GPU Computing Series, W. mei W. Hwu, Ed. Boston: Morgan Kaufmann, 2012, pp. 359–371. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780123859631000265